



Universidad Técnica Nacional

Sede Regional San Carlos

Ingeniería del Software

Programación en Ambiente Web I, ISW-521, II-26, SC-03

Investigación: Vite vs Webpack

Docente,

Bryan Salas Chavez

Estudiantes,

Fabián Zamora,

Giovanni Sandi

Fecha de entrega,

30/07/2026

Tabla de contenidos

1. Resumen Ejecutivo	1
1.1 Idea Central.....	1
1.2 Hallazgos claves	1
2. Definición Técnica: ¿Que Son y que Problema Resuelven?	1
2.1 Webpack	1
2.2 Vite	1
3. Funcionamiento interno.....	1
3.1 Webpack: grafo de dependencias, loaders y plugins	1
3.2 Vite: ESM nativo , pre-building y Rollup/Rolldown.....	1
4. ¿Cuándo conviene usar cada una?	1
4.1 Uso de Vite.....	1
4.2 Uso de Webpack.....	1
4.3 Casos donde no se deben de usar	1
5. Comparativa técnica directa.....	1
6. Estado de adopción en la Industria (2025-2026)	1
6.1 Encuestas de la industria: State of JavaScript.....	1
6.2 Decargas npm y métricas de repositorio.....	1
6.3 Frameworks y meta-frameworks	1
7. Limitaciones y Desventajas en Producción	1
7.1 Limitaciones de Vite.....	1
7.2 Limitaciones de Webpack.....	1
8. Seguridad y Rendimiento en Producción	1
8.1 Consideraciones de seguridad	1
8.2 Rendimiento en producción.....	1
9. Recomendaciones y Conclusiones.....	1
Bibliografía	1

1. Resumen Ejecutivo

1.1 Idea Central

Webpack es un bundler de módulos maduro y altamente configurable que empaqueta toda la aplicación en un grafo de dependencias antes de servirla. Vite es una herramienta de build de nueva generación que, en desarrollo, sirve el código fuente sin empaquetar mediante módulos ES (ESM) nativos del navegador, y solo empaqueta para producción. Para proyectos nuevos (React, Vue, Svelte), Vite es hoy la elección por defecto de la industria; Webpack sigue siendo defendible en escenarios legacy o que requieran Module Federation madura. La diferencia central es arquitectónica: Webpack construye el bundle completo antes de servir (arranque en frío que crece con el tamaño del proyecto), mientras Vite tiene un arranque casi constante e independiente del tamaño, gracias al pre-empaquetado de dependencias con esbuild y al Hot Module Replacement (HMR) sobre ESM nativo. Según la cobertura de InfoQ sobre la encuesta State of JavaScript 2025, Vite alcanzó un 98 % de satisfacción de sus usuarios, mientras Webpack cayó a 26 % (desde 36 % en 2024). La principal desventaja técnica histórica de Vite era la inconsistencia entre desarrollo y producción, al usar dos motores de bundling distintos (esbuild/ESM en dev, Rollup en build). Esto se resolvió arquitectónicamente en Vite 8 (12 de marzo de 2026) al unificar ambos entornos sobre Rolldown, un bundler en Rust. La principal desventaja de Webpack sigue siendo la complejidad de configuración y la degradación de rendimiento en proyectos grandes.

1.2 Hallazgos claves

- Webpack resuelve el problema de empaquetar múltiples módulos (JS, CSS, imágenes, fuentes) en pocos bundles cargables por el navegador, mediante loaders (transforman archivos) y plugins (extienden el proceso de build).
- Vite resuelve el problema de la lentitud de arranque y HMR de los bundlers tradicionales, sirviendo ESM nativo en desarrollo y delegando el bundling de producción a Rollup/Rolldown.
- En desarrollo, Webpack construye todo el bundle antes de servir; Vite no empaqueta el código fuente y transforma bajo demanda, por lo que su arranque en frío es casi constante sin importar el tamaño de la app.
- Vite pre-empaqueta dependencias con esbuild (escrito en Go): según la documentación oficial, esbuild «pre-bundles dependencies 10-100x faster than JavaScript-based bundlers».
- La adopción se ha inclinado decisivamente hacia Vite en 2025-2026 (98 % de satisfacción vs. 26 % de Webpack en State of JS 2025), aunque Webpack aún mantiene una ligera ventaja en uso absoluto por su base instalada histórica.

- Vite 8 (marzo 2026) reemplazó tanto a esbuild como a Rollup por Rolldown, un bundler unificado en Rust, eliminando la inconsistencia entre desarrollo y producción.

2. Definición Técnica: ¿Que Son y que Problema Resuelven?

2.1 Webpack

Según su documentación oficial, en su núcleo, webpack es un static module bundler para aplicaciones JavaScript modernas. Cuando webpack procesa tu aplicación, internamente construye un grafo de dependencias a partir de uno o más entry points y luego combina cada módulo que tu proyecto necesita en uno o más bundles, que son assets estáticos para servir tu contenido. Webpack nació en 2012 y su problema concreto es el empaquetado: antes de la llegada de los módulos ES nativos, el navegador no tenía un mecanismo estándar para cargar módulos JavaScript de forma modular. Webpack extiende este concepto para tratar CSS, imágenes y fuentes también como módulos, mediante loaders.

2.2 Vite

Según la documentación oficial de Vite (Why Vite), los desarrolladores en proyectos grandes «han experimentado arranques de servidor de desarrollo dolorosamente lentos, actualizaciones en caliente perezosas y tiempos largos de build de producción». Vite (del francés rápido) fue creado por Evan You (creador de Vue) y lanzado en 2020 para resolver este problema repensando cómo debe servirse el código durante el desarrollo, en lugar de mejorar incrementalmente los enfoques existentes. Su problema concreto es el cuello de botella de rendimiento del tooling basado en JavaScript, donde arrancar un dev servidor podía tomar minutos en proyectos grandes.

3. Funcionamiento interno

3.1 Webpack: grafo de dependencias, loaders y plugins

- **Entry points y grafo de dependencias:** Webpack comienza en uno o más módulos de entrada (por defecto `./src/index.js`) y construye recursivamente un grafo de dependencias con cada módulo que la aplicación necesita, empaquetándolos en un número reducido de bundles. El proceso es: (1) empezar en los entry points; (2) parsear cada módulo extrayendo sentencias `import/require`; (3) resolver las dependencias localizando los archivos importados; (4) añadirlas al grafo mediante ordenamiento topológico.
- **Loaders. Por defecto, Webpack solo entiende JavaScript y JSON:** Los loaders permiten procesar otros tipos de archivo y convertirlos en módulos válidos que se añaden al grafo de dependencias. Operan por archivo y se ejecutan de derecha a izquierda (ej. `sass-loader` → `css-loader` → `style-loader`).
- **Plugins:** Realizan tareas más amplias que los loaders (optimización del bundle, gestión de assets, inyección de variables de entorno). Webpack posee una arquitectura orientada a eventos: un plugin es un objeto con un método `apply` que registra hooks en el ciclo de vida del compilador. El propio núcleo de Webpack está construido sobre su sistema de plugins.
- **Output, optimización, dev server y HMR:** Webpack emite los bundles a `./dist` por defecto. En modo producción activa tree shaking (eliminación de código muerto vía análisis estático de ESM), code splitting, minificación (Terser), scope hoisting y hashing de contenido para caching a largo plazo. El `webpack-dev-server` compila el bundle en memoria antes de servirlo; el HMR se gestiona con el `HotModuleReplacementPlugin` a través de la interfaz `module.hot`. Si un módulo no tiene manejadores HMR, la actualización burbujea hacia arriba; si falla, se recarga la página completa.

3.2 Vite: ESM nativo , pre-building y Rollup/Rolldown

- **ESM nativo en desarrollo, sin bundling:** La documentación de Vite es explícita: «la diferencia principal es que para Vite no hay bundling durante el desarrollo. La sintaxis de import ES en el código fuente se sirve directamente al navegador, que la parsea mediante soporte nativo de <script module>, haciendo una petición HTTP por cada import». El dev server intercepta esas peticiones y transforma el código al vuelo (por ejemplo, un archivo .vue o .tsx) justo antes de enviarlo. Como no hay trabajo de bundling, el arranque en frío es extremadamente rápido y el código se compila bajo demanda solo lo importado en la pantalla actual.
- **Pre-bundling de dependencias:** La primera vez que se ejecuta vite, se pre-empaquetan las dependencias de node_modules con dos propósitos: (1) compatibilidad CommonJS/UMD convertirlas a ESM, ya que en dev todo se sirve como ESM nativo; y (2) rendimiento convertir dependencias ESM con muchos módulos internos en un solo módulo. El ejemplo canónico de la documentación oficial: lodash-es tiene más de 600 módulos internos; sin pre-bundling, un simple `import { debounce } from 'lodash-es'` dispararía 600+ peticiones HTTP simultáneas. El pre-bundling se hace con esbuild (Go), «10-100x más rápido que los bundlers basados en JavaScript» según la documentación oficial.
- **HMR sobre ESM nativo:** Cuando se edita un archivo, Vite invalida con precisión solo la cadena entre el módulo editado y su HMR boundary más cercano la mayoría de las veces, solo el módulo mismo, haciendo que las actualizaciones sean consistentemente rápidas sin importar el tamaño de la aplicación. Esta es la diferencia arquitectónica clave frente a Webpack: la velocidad de HMR está desacoplada del número total de módulos del proyecto
- **Rollup / Rolldown para producción:** Históricamente, vite build usaba Rollup para empaquetar producción, mientras esbuild se usaba solo en dev —una arquitectura de dos motores que generaba inconsistencias sutiles entre ambos entornos. El blog oficial «Vite 8.0 is out!» (12 de marzo de 2026) anunció la solución: Vite 8 ships with Rolldown as its single, unified, Rust-based bundler, delivering up to 10-30x faster builds while maintaining full plugin compatibility. This is the most significant architectural change since Vite 2. Los transforms se delegan además a Oxc, un toolchain también escrito en Rust.

4. ¿Cuándo conviene usar cada una?

4.1 Uso de Vite

- El proyecto es nuevo y usa React, Vue, Svelte, Solid, Lit o vanilla JS/TS. Vue 3 usa Vite como scaffolding oficial; SvelteKit, Astro, Nuxt 3/4, Laravel, SolidStart y Remix/React Router 7 lo usan por defecto.
- Se prioriza la velocidad de desarrollo (arranque instantáneo, HMR rápido) y una configuración mínima. El propio equipo de React, al anunciar el fin de Create React App (14 de febrero de 2025), recomendó explícitamente migrar «a un framework, o a una herramienta de build como Vite, Parcel o RSBUILD».
- Se construyen librerías de componentes o design systems, aprovechando el modo librería de Vite (sobre Rollup/Rolldown), que emite ESM y UMD en una sola pasada.

4.2 Uso de Webpack

- Existe un proyecto legacy grande con configuración Webpack madura y funcional, donde el riesgo/costo de migración no se justifica frente al beneficio.
- Se requiere Module Federation madura para micro-frontends. El caso canónico es Discord: su cliente web usa Module Federation de Webpack 5 para micro-frontends versionados de forma independiente. El plugin equivalente para Vite (@originjs/vite-plugin-federation) existe, pero no está a paridad para casos complejos.
- Se depende de loaders o plugins muy específicos (o DLL plugins) sin equivalente maduro en el ecosistema de Vite.

4.3 Casos donde no se deben de usar

- **Webpack** es la herramienta equivocada para iniciar un proyecto nuevo simple donde la velocidad de feedback importa: existe consenso amplio de que no hay razón sólida para arrancar un proyecto React o Vue nuevo con Webpack en 2026.
- **Vite** «puro» es la herramienta equivocada, por ejemplo, para Next.js, que tiene su propio tooling: desde Next.js 16 (octubre 2025) usa Turbopack por defecto tanto en desarrollo como en producción, con Webpack disponible solo como fallback explícito (--webpack). De forma similar, Angular migró su CLI a esbuild + Vite (este último únicamente como dev server) desde las versiones 17/18, dejando en desuso el builder basado en Webpack.

5. Comparativa técnica directa

Dimensión	Vite	Webpack
Arranque en frío (dev)	Casi constante e independiente del tamaño del proyecto (no empaqueta).	Crece con el tamaño del proyecto; debe empaquetar todo antes de servir.
Hot Module Replacement	Sobre ESM nativo; invalida solo el módulo editado; velocidad desacoplada del nº de módulos.	Vía HotModuleReplacementPlugin; puede requerir re-bundle parcial; se degrada en apps grandes.
Filosofía de configuración	Convención sobre configuración; muchos proyectos sin archivo de config o con uno mínimo.	Altamente configurable; archivos de configuración extensos; curva de aprendizaje pronunciada.
Ecosistema de plugins	Compatible con la API de plugins de Rollup (miles de plugins); catálogo propio en crecimiento.	El más maduro del ecosistema; más de una década de loaders y plugins.
TypeScript	Type stripping vía esbuild/Oxc (sin chequeo de tipos); requiere tsc aparte o vite-plugin-checker.	Vía ts-loader o babel-loader; ts-loader puede hacer chequeo de tipos durante el build.
CSS preprocessors	Soporte integrado de Sass, Less, Stylus, PostCSS y CSS Modules, con HMR.	Vía loaders encadenados (sass-loader, css-loader, style-loader, postcss-loader).
Code splitting	Automático sobre import() dinámico, vía Rollup/Rolldown.	Manual y automático vía SplitChunksPlugin; muy sofisticado y configurable.

Tree shaking	Vía Rollup/Rolldown, con buen scope hoisting.	Vía análisis estático de ESM; no funciona con AMD/UMD; requiere declarar sideEffects.
Navegadores legacy / IE	Target mínimo es ESM nativo; requiere @vitejs/plugin-legacy (Babel + core-js + SystemJS).	Puede targetear ES5 nativamente vía Babel; más directo para IE11 sin plugins especiales.
Versión estable (jul. 2026)	Vite 8.x (Vite 8.0 estable desde el 12 de marzo de 2026).	webpack 5.108.4 (julio de 2026).

6. Estado de adopción en la Industria (2025-2026)

6.1 Encuestas de la industria: State of JavaScript

En State of JS 2024, Webpack fue la herramienta más usada (85,3 %), pero Vite fue la más querida por sus usuarios (56 % de opiniones positivas) y ya la tercera más usada (78,1 %), con una retención cercana al 72-75 %.

En State of JS 2025 (encuesta de Devographics realizada en noviembre de 2025, publicada en 2026 y patrocinada por Google Chrome y JetBrains), Vite alcanzó aproximadamente 98 % de satisfacción, mientras Webpack cayó a 26 % (desde 36 % en 2024). En términos de uso, Webpack mantiene todavía una ligera ventaja (86,4 % frente a 84,4 % de Vite, una brecha de apenas 2 puntos). La queja número uno sobre herramientas de build en la encuesta fue la complejidad de configuración (señalada explícitamente por 289 personas encuestadas). Rolldown saltó de 1 % a 10 % de uso y lidera en «interés» entre las nuevas herramientas.

6.2 Descargas npm y métricas de repositorio

Vite superó a Webpack en descargas semanales de npm en julio de 2025 por primera vez. El blog oficial de Vite (12 de marzo de 2026) reportó que Vite is now being downloaded 65 million times a week. Los tableros de npm trends a mediados de 2026 muestran cifras aún mayores para el paquete vite (del orden de 120-147 millones semanales para Vite 8.x) frente a aproximadamente 45-48 millones de webpack 5.x conviene distinguir la cifra oficial conservadora del lanzamiento (65M) de las cifras de npm trends, que crecen de forma continua. En GitHub, Vite acumula cerca de 81 000 estrellas frente a aproximadamente 65 700 de Webpack.

6.3 Frameworks y meta-frameworks

Usan Vite por defecto	Dependen de Webpack o sucesores
Vue 3, Nuxt 3/4, SvelteKit, Astro, Laravel, SolidStart, Remix / React Router 7, Shopify Hydrogen. El dev server de Angular CLI usa Vite desde la v17.	Next.js migró a Turbopack (sucesor en Rust de Vercel) por defecto desde Next.js 16 (octubre 2025); GitLab aún envía su monolito Rails con Webpack 5; algunos micro-frontends que dependen de Module Federation.

Versiones estables vigentes (julio de 2026): webpack 5.108.4; Vite 8.1.x (Vite 8.0 estable desde el 12 de marzo de 2026, con Rolldown 1.0 estable desde el 7 de mayo de 2026, API congelada y garantías de retrocompatibilidad). Vite 7+ requiere Node.js 20.19+ o 22.12+, al distribuirse únicamente como ESM.

7. Limitaciones y Desventajas en Producción

7.1 Limitaciones de Vite

- ♦ **Inconsistencia dev/prod (histórica):** Al usar esbuild/ESM en dev y Rollup en build, podían aparecer diferencias sutiles: manejo distinto de dependencias CommonJS, comportamiento distinto de code splitting o de side effects. El bug clásico era un import por defecto de CommonJS que resolvía bien en dev y devolvía undefined en el bundle de producción, por diferencias de interoperabilidad entre esbuild y Rollup. Vite 8 (marzo 2026) resolvió esto arquitectónicamente al unificar dev y build sobre Rolldown.
- ♦ **Network waterfalls en desarrollo:** Los imports ESM nativos con cadenas profundas de dependencias producen cascadas de peticiones HTTP; el recargado completo de página puede ser algo más lento que en un setup basado en bundler tradicional, aunque el costo es trivial en desarrollo local al estar cacheado en memoria.
- ♦ Los **builds de producción en aplicaciones muy grandes** podían históricamente ser más lentos que setups Webpack muy optimizados brecha que se reduce sensiblemente con Rolldown. El ecosistema de plugins es más pequeño que el de Webpack; algunos casos límite legacy pueden requerir soluciones alternativas (workarounds).

7.2 Limitaciones de Webpack

- ◇ **Lentitud:** Los tiempos de arranque del dev server y de build crecen con el tamaño del proyecto; en proyectos grandes puede tomar decenas de segundos arrancar. La literatura de troubleshooting empresarial coincide en que el tiempo de rebuild crece con el proyecto hasta convertirse en un problema real para el ciclo de desarrollo y release.
- ◇ **Complejidad de configuración ("config hell"):** Al ser tan abierto y potente, la configuración puede volverse muy compleja, con documentación descrita frecuentemente como densa y una curva de aprendizaje pronunciada; fue la queja número uno en State of JS 2025.
- ◇ **Consumo de memoria:** Webpack empaqueta en memoria; grafos de módulos grandes pueden exceder el heap de Node y requerir ajustar `--max-old-space-size`.
- ◇ **Hashes no deterministas** derivados de plugins mal configurados (por ejemplo, el orden de extracción de CSS) pueden afectar negativamente el caching a largo plazo.
- ◇ **Tree shaking limitado con Module Federation:** los módulos declarados como shared se incluyen completos, un comportamiento documentado como issue abierto en el propio repositorio de Webpack.

8. Seguridad y Rendimiento en Producción

8.1 Consideraciones de seguridad

Vulnerabilidades de Vite del dev server (2025-2026). Se han reportado varias vulnerabilidades de lectura arbitraria de archivos en el dev server de Vite:

- **CVE-2025-30208:** (GitHub Advisory GHSA-x574-m823-4x7w, publicado el 24 de marzo de 2025): permitía sortear la restricción `server.fs.deny` añadiendo `?raw??` o `?import&raw??` a URLs con el prefijo `@fs`, posibilitando la lectura de archivos arbitrarios del sistema. El propio aviso aclara: «Only apps explicitly exposing the Vite dev server to the network (using `--host` or `server.host` config option) are affected». Parchado en las versiones 6.2.3, 6.1.2, 6.0.12, 5.4.15 y 4.5.10.
- **CVE-2026-39363:** (GHSA-p9ff-h696-f583): lectura arbitraria de archivos vía el WebSocket del dev server. Un atacante conectado al WebSocket sin cabecera Origin podía invocar `fetchModule` mediante el evento `vite:invoke` y combinar el prefijo `file://` con los parámetros `?raw` o `?inline` para leer archivos arbitrarios, ya que la verificación de `server.fs` no se aplicaba a esa ruta de ejecución. Parchado en las versiones 6.4.2, 7.3.2 y 8.0.5.

La mitigación clave en ambos casos es la misma: nunca exponer el dev server de Vite a internet público (el riesgo solo se activa con las opciones `--host / server.host`); el dev server no está diseñado para producción.

Para Webpack la propia documentación oficial advierte que el HMR no está pensado para uso en producción. Los riesgos de seguridad en producción tienen más que ver con el código empaquetado y sus dependencias que con el bundler en sí mismo; se recomienda, por ejemplo, no exponer información sensible mediante nombres de módulo con opciones como `chunkIds: 'named'`.

En ambos casos, el output de producción debe considerar minificación adecuada, evitar incluir source maps sensibles en despliegues públicos, y vigilar la cadena de suministro de dependencias npm.

8.2 Rendimiento en producción

Vite (sobre Rollup/Rolldown) tiende a producir bundles más pequeños y limpios que Webpack, gracias a un mejor tree-shaking y scope hoisting. Vite usa Oxc o esbuild para minificación, muy por encima en velocidad de Terser: la documentación oficial indica que el minificador Oxc por defecto es «30~90x más rápido que terser y solo 0,5~2 % peor en compresión»; para CSS, el minificador por defecto es Lightning CSS.

El benchmark oficial publicado por el propio proyecto esbuild (empaquetando three.js diez veces) muestra a esbuild completando la tarea en 0,37s frente a aproximadamente 42,9s de Webpack 5 cerca de 116 veces más lento; es un benchmark del propio esbuild, con metodología pública disponible para revisión.

Sobre Rolldown, el blog oficial de Vite y la cobertura de InfoQ (14 de mayo de 2026, «Vite Version 8») reportan casos de estudio publicados por VoidZero durante la fase beta: Linear saw production build times drop from 46 seconds to 6 seconds, Ramp reported a 57% reduction, and Beehiiv achieved a 64% improvement (Mercedes-Benz.io reportó hasta -38 %).

Por su parte, Webpack 5 introdujo caching persistente en disco, hashes basados en el contenido real del archivo ([contenthash]) para caching a largo plazo, y mejoras al tree shaking respecto a Webpack 4.

9. Recomendaciones y Conclusiones

- Para proyectos nuevos (incluyendo el propio proyecto demo de esta investigación): usar Vite con la plantilla del framework correspondiente (`npm create vite@latest`). Justificación: arranque instantáneo, HMR rápido y configuración mínima, siendo además el estándar recomendado incluso por el equipo de React tras deprecarse Create React App en febrero de 2025.
- Si el proyecto es un monolito legacy con Webpack funcional y Module Federation en producción: mantener Webpack, o evaluar Rspack (bundler compatible con Webpack, escrito en Rust) como paso intermedio de migración. Migrar por completo solo si el equipo sufre activamente arranques lentos del dev server.
- Si el proyecto usa Next.js: no elegir Vite ni Webpack de forma directa; usar el tooling integrado del framework (Turbopack por defecto desde Next.js 16).
- Umbrales orientativos para decidir migrar de Webpack a Vite: si el arranque del dev server supera entre 10 y 15 segundos y afecta la productividad del equipo, si no se depende de Module Federation compleja, y si las dependencias del proyecto son mayormente ESM.
- Seguridad operativa (ambas herramientas): nunca exponer el dev server a internet; mantener versiones parchadas (Vite $\geq$ 8.0.5 / 7.3.2 / 6.4.2 por los CVEs de 2026); no desplegar HMR ni source maps sensibles en producción.

Bibliografía

Angular. (s. f.). *Migrating to the new build system*. Documentación oficial.

<https://angular.dev/tools/cli/build-system-migration>

Devographics. (2024). *State of JavaScript 2024*. <https://2024.stateofjs.com/>

Devographics. (2025). *State of JavaScript 2025*. <https://2025.stateofjs.com/>

esbuild. (s. f.). *Documentación y benchmarks oficiales*. <https://esbuild.github.io/>

GitHub. (s. f.). *Repositorio oficial vitejs/vite (releases y changelog)*.

<https://github.com/vitejs/vite>

GitHub. (s. f.). *Repositorio oficial webpack/webpack (releases y changelog)*.

<https://github.com/webpack/webpack>

GitHub Advisory Database. (2025, 24 de marzo). *GHSA-x574-m823-4x7w — CVE-2025-30208 (Vite arbitrary file read)*. <https://github.com/advisories/GHSA-x574-m823-4x7w>

GitHub Advisory Database. (2026). *GHSA-p9ff-h696-f583 — CVE-2026-39363 (Vite dev server WebSocket arbitrary file read)*. <https://github.com/advisories/GHSA-p9ff-h696-f583>

InfoQ. (2026, marzo). *State of JavaScript 2025: Survey reveals a maturing ecosystem with TypeScript cementing dominance*. <https://www.infoq.com/news/2026/03/state-of-js-survey-2025/>

InfoQ. (2026, 14 de mayo). *Vite version 8*. <https://www.infoq.com/news/2026/05/vite-8/>

Next.js. (s. f.). *Turbopack*. Documentación oficial. <https://nextjs.org/docs/app/api-reference/turbopack>

Next.js. (2025). *Upgrading: Version 16*. Documentación oficial.

<https://nextjs.org/docs/app/guides/upgrading/version-16>

npm trends. (s. f.). *Comparativa de descargas semanales: vite vs. webpack*.

<https://npmtrends.com/vite-vs-webpack>

React. (2025, 14 de febrero). *Sunsetting Create React App*. Blog oficial de React.

<https://react.dev/blog/2025/02/14/sunsetting-create-react-app>

Vite. (s. f.). *Build options*. Documentación oficial. <https://vite.dev/config/build-options>

Vite. (s. f.). *Dependency pre-bundling*. Documentación oficial. <https://vite.dev/guide/dep-pre-bundling>

Vite. (s. f.). *Features*. Documentación oficial. <https://vite.dev/guide/features>

Vite. (s. f.). *Why Vite*. Documentación oficial. <https://vite.dev/guide/why>

Vite Team. (2025). *Vite 7.0 is out!* Blog oficial de Vite. <https://vite.dev/blog/announcing-vite7>

Vite Team. (2026, 12 de marzo). *Vite 8.0 is out!* Blog oficial de Vite. <https://vite.dev/blog/announcing-vite8>

VoidZero. (s. f.). *Documentación oficial de Rolldown*. <https://voidzero.dev/>

webpack. (s. f.). *Concepts*. Documentación oficial. <https://webpack.js.org/concepts/>

webpack. (s. f.). *Dependency graph*. Documentación oficial. <https://webpack.js.org/concepts/dependency-graph/>

webpack. (s. f.). *Hot module replacement*. Documentación oficial. <https://webpack.js.org/concepts/hot-module-replacement/>

webpack. (s. f.). *To v5 from v4*. Guía de migración oficial. <https://webpack.js.org/migrate/5/>